



API del proyecto.

GA7-220501096-AA5-EV04

FICHA 3186658

APRENDIZ

STEPHANY ROMERO TORO

PRESENTADO AL INSTRUCTOR

DIEGO MANUEL DUSSAN MUÑOZ

ANÁLISIS DESARROLLO DE SOFTWARE

SERVICIO NACIONAL DE APRENDIZAJE SENA

REGIONAL HUILA

JULIO 2026

Introducción

El presente documento describe el proceso de testing realizado a la API desarrollada en Node.js utilizando el framework Express. Para la validación de los servicios implementados se empleó la herramienta Postman, permitiendo verificar el correcto funcionamiento de los endpoints encargados del registro y autenticación de usuarios. Las pruebas realizadas incluyeron tanto escenarios exitosos como casos de error, con el fin de comprobar que la API responde de manera adecuada.

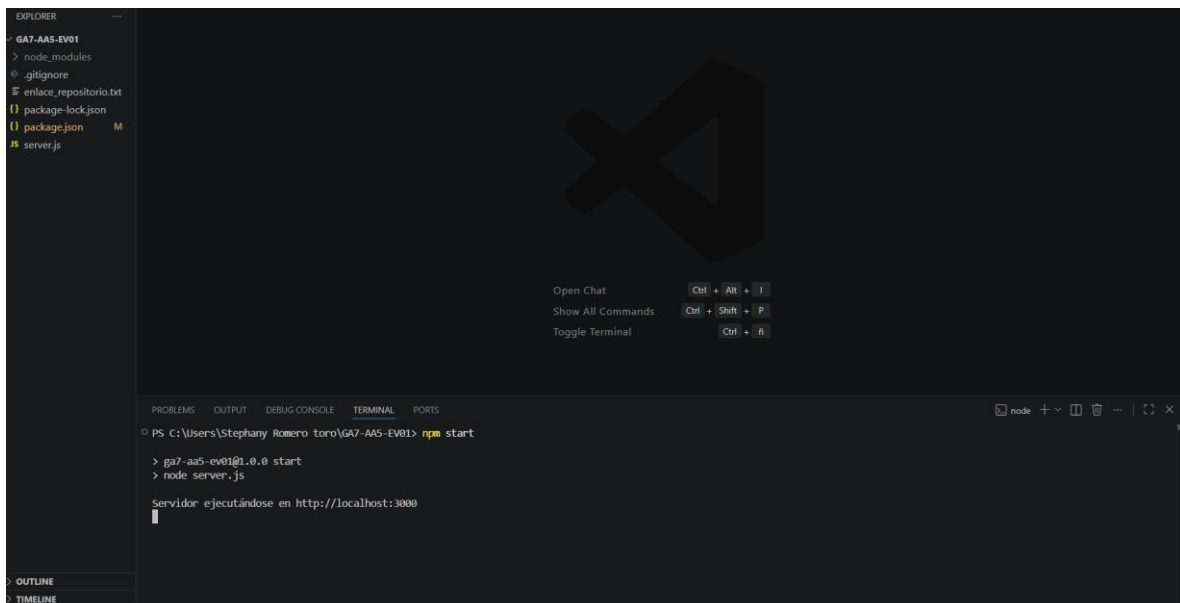
Objetivo

Realizar pruebas funcionales a la API desarrollada mediante la herramienta Postman, verificando el correcto funcionamiento de los servicios implementados para el registro y autenticación de usuarios, así como la correcta respuesta del servidor ante solicitudes válidas e inválidas.

Resultados de las pruebas

Las pruebas fueron ejecutadas utilizando el host local:

Host utilizado: <http://localhost:3000>



```
EXPLORER
- GA7-AA5-EV01
  > node_modules
  > .gitignore
  > enlace_repositorio.txt
  > package-lock.json
  > package.json
  > server.js

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Stephany Romero\toro\GA7-AA5-EV01> npm start
> ga7-aa5-ev01@1.0.0 start
> node server.js

Servidor ejecutándose en http://localhost:3000
```

Durante el proceso de validación se comprobó que el servidor respondió correctamente a las solicitudes HTTP realizadas desde Postman. Asimismo, se verificó el funcionamiento de cada uno de los endpoints implementados, confirmando que la API registra nuevos

usuarios, autentica credenciales válidas y devuelve los códigos de respuesta apropiados cuando se presentan errores de autenticación.

Pruebas realizadas

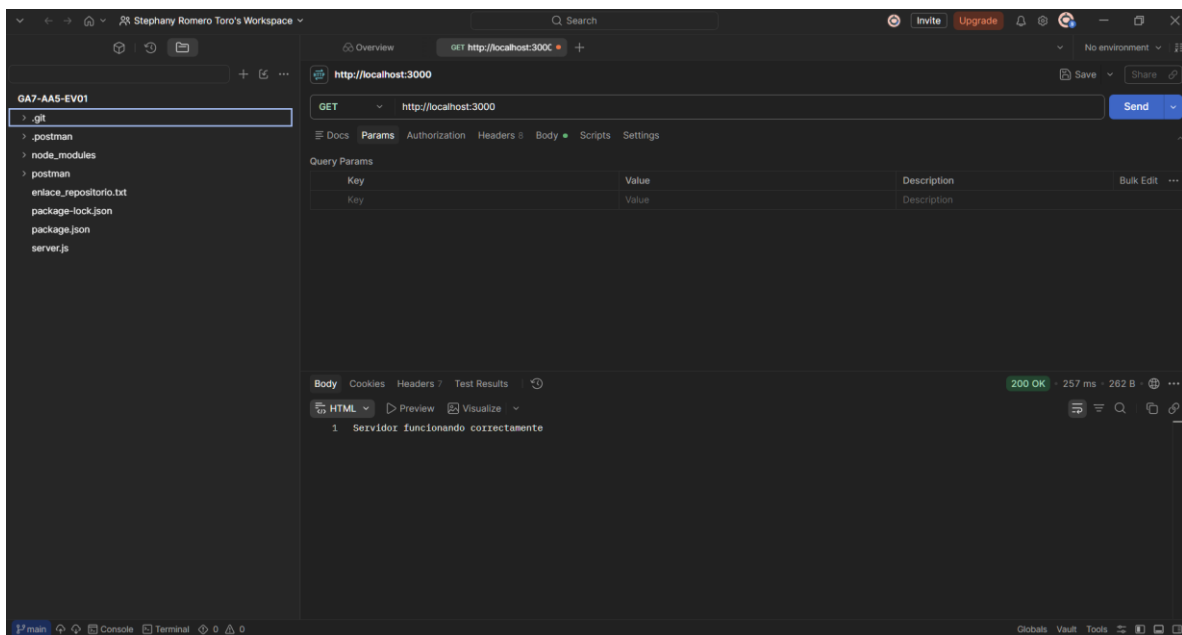
Prueba 1. Verificación del funcionamiento del servidor

Se realizó una solicitud HTTP de tipo **GET** al endpoint principal `http://localhost:3000/`, con el propósito de verificar que el servidor estuviera en ejecución y disponible para recibir solicitudes.

Resultado obtenido

La API respondió con el mensaje:

Servidor funcionando correctamente



Prueba 2. Registro de usuario

Se realizó una solicitud HTTP de tipo **POST** al endpoint `http://localhost:3000/register`, enviando en el cuerpo de la petición un objeto JSON con la información del usuario.

Datos enviados

```
{  
  
  "usuario": "stephany",
```

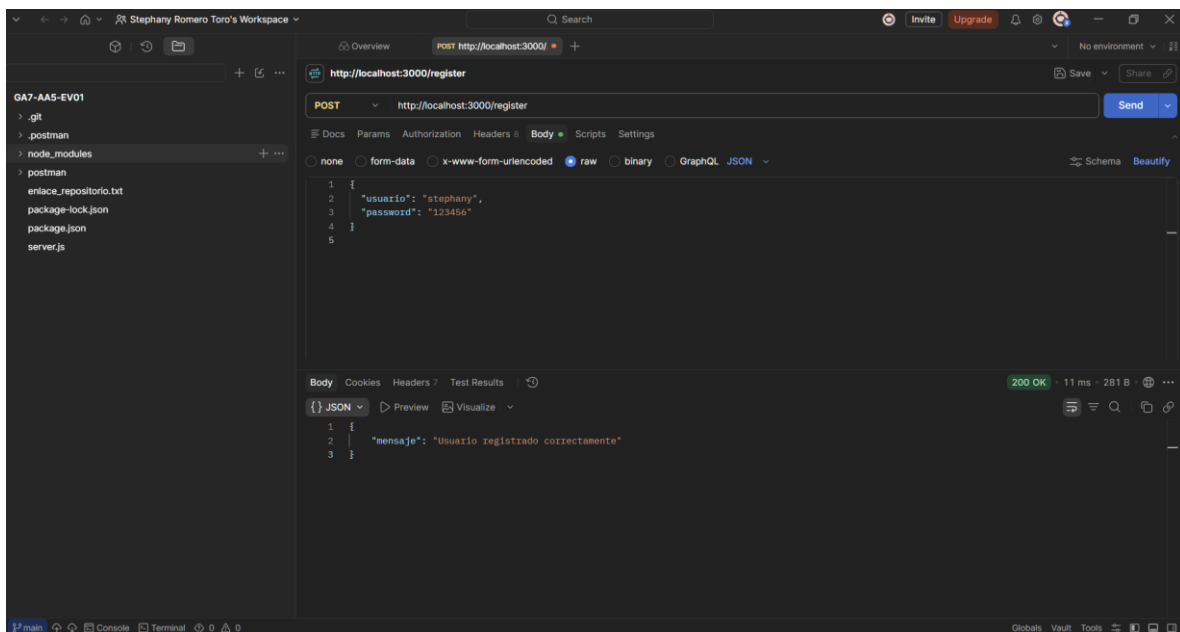
```
"password": "123456"
}
```

Resultado obtenido

La API respondió con el siguiente mensaje:

```
{
  "mensaje": "Usuario registrado correctamente"
}
```

Este resultado confirma que el servicio registró correctamente la información recibida.



Prueba 3. Inicio de sesión exitoso

Se realizó una solicitud HTTP de tipo **POST** al endpoint `http://localhost:3000/login`, utilizando las mismas credenciales registradas anteriormente.

Datos enviados

```
{
  "usuario": "stephany",
  "password": "123456"
}
```

```
}
```

Resultado obtenido

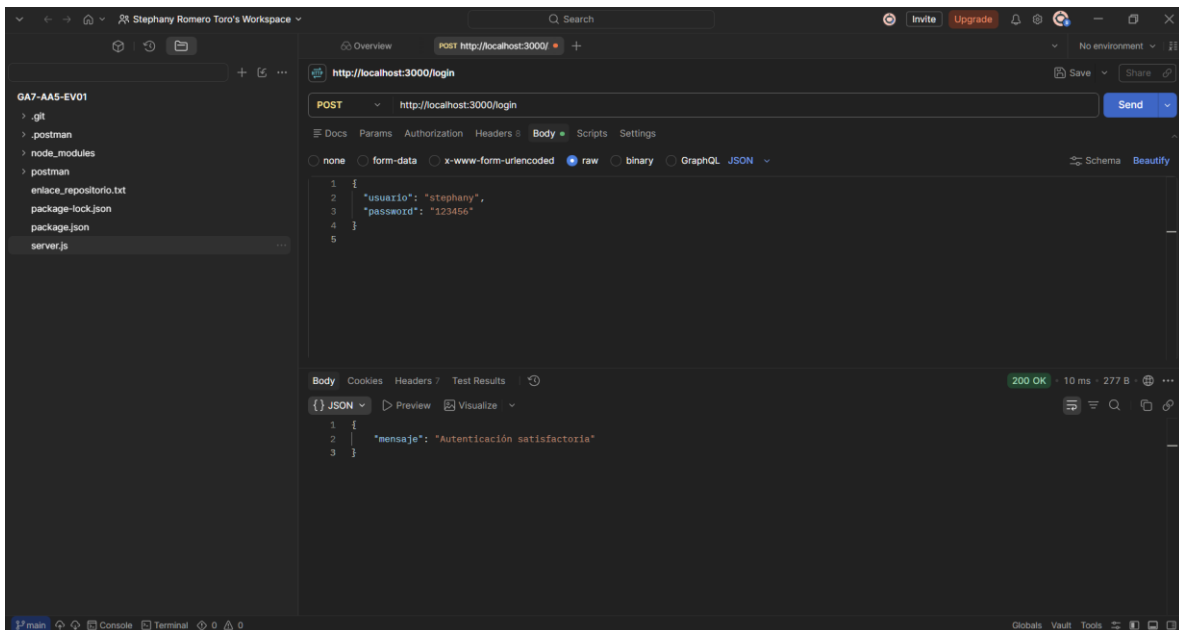
La API respondió con el siguiente mensaje:

```
{
```

```
  "mensaje": "Autenticación satisfactoria"
```

```
}
```

Con esta prueba se verificó que el sistema autentica correctamente a los usuarios cuando las credenciales son válidas.



Prueba 4. Validación de credenciales incorrectas

Finalmente, se realizó una nueva solicitud HTTP de tipo **POST** al endpoint `http://localhost:3000/login`, ingresando una contraseña diferente a la registrada.

Datos enviados

```
{
```

```
  "usuario": "stephany",
```

```
  "password": "111111"
```

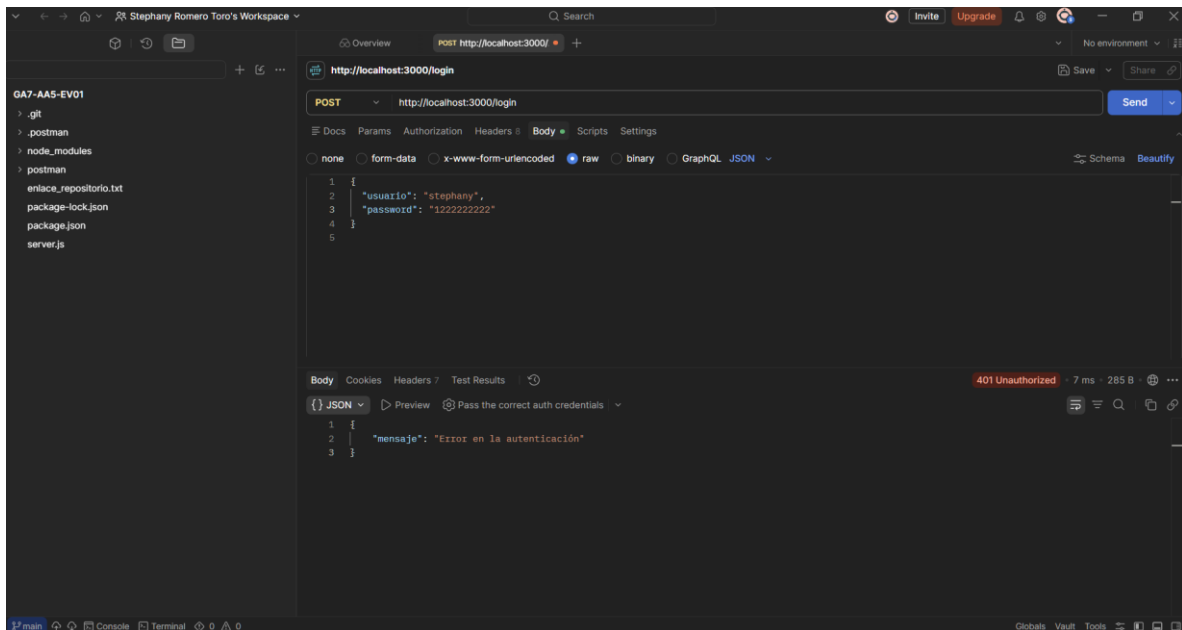
```
}
```

Resultado obtenido

La API respondió con el código de estado **401 (Unauthorized)** y el siguiente mensaje:

```
{
  "mensaje": "Error en la autenticación"
}
```

Esta respuesta demuestra que la API valida correctamente las credenciales y restringe el acceso cuando la información suministrada no coincide con la registrada.



Endpoints de la API

La API desarrollada cuenta con tres endpoints principales, los cuales fueron validados mediante la herramienta Postman. Durante las pruebas se verificó el comportamiento esperado de cada servicio, incluyendo los diferentes escenarios de respuesta del endpoint de autenticación.

Método HTTP	Endpoint	Descripción	Resultado esperado

GET	http://localhost:3000/	Verifica que el servidor se encuentre en funcionamiento.	200 OK –Mensaje: <i>"Servidor funcionando correctamente"</i> .
POST	http://localhost:3000/register	Permite registrar un nuevo usuario enviando la información en formato JSON.	200 OK – Mensaje: <i>"Usuario registrado correctamente"</i> .
POST	http://localhost:3000/login	Permite autenticar un usuario con las credenciales registradas.	200 OK – Mensaje: <i>"Autenticación satisfactoria"</i> cuando las credenciales son válidas.
POST	http://localhost:3000/login	Validación de credenciales incorrectas utilizando el mismo endpoint de autenticación.	401 Unauthorized – Mensaje: <i>"Error en la autenticación"</i> cuando las credenciales son inválidas.

Conclusiones

- Se comprobó que el servidor se inicia correctamente y responde a las solicitudes realizadas mediante la herramienta Postman.
- El endpoint **GET /** permitió verificar la disponibilidad de la API y confirmar que el servicio se encontraba operativo.
- El endpoint **POST /register** registró correctamente un nuevo usuario a partir de la información enviada en formato JSON, demostrando el correcto procesamiento de los datos recibidos.
- El endpoint **POST /login** autenticó exitosamente al usuario cuando se utilizaron credenciales válidas y, adicionalmente, respondió con el código de estado **401**

(Unauthorized) cuando se ingresaron credenciales incorrectas, evidenciando un adecuado manejo de errores.

- Las pruebas funcionales realizadas permitieron validar el comportamiento esperado de todos los endpoints implementados, confirmando que la API responde correctamente tanto en escenarios exitosos como en situaciones de autenticación fallida.